四轮全向底底盘运动学分析

> 基于45°布局的四轮全向轮（Omni Wheel）底盘运动学建模与ROS实现。

一、底盘结构

1.1 轮子布局

轮子安装方向（以x轴为起点，逆时针为正）：

- **轮1（左前）**：45°
- **轮2（左后）**：135°
- **轮3（右后）**：-135°（225°）
- **轮4（右前）**：-45°（315°）

```
      前进x轴方向 ↑

轮1 ↗          ↖ 轮4
        ┌──────┐
        │      │
←       │  •   │ y轴
        │      │
        └──────┘
轮2 ↘          ↙ 轮3
```

1.2 关键参数

参数	符号	说明
底盘边长	$L_{chassis}$	正方形底盘
轮子半径	R	全向轮半径
轮心到中心距离	L	$L = \frac{L_{chassis}}{\sqrt{2}}$

二、运动学模型

2.1 逆向运动学（速度 → 轮速）

输入：底盘目标速度 (v_x, v_y, ω_z)
输出：四个轮子的线速度 (V_1, V_2, V_3, V_4)

矩阵形式：

$$\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 & \sqrt{2}L \\ -1 & 1 & \sqrt{2}L \\ -1 & -1 & \sqrt{2}L \\ 1 & -1 & \sqrt{2}L \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix}$$

展开形式：

$$\begin{cases} V_1 = \frac{1}{\sqrt{2}}(v_x + v_y) + L\omega_z \\ V_2 = \frac{1}{\sqrt{2}}(-v_x + v_y) + L\omega_z \\ V_3 = \frac{1}{\sqrt{2}}(-v_x - v_y) + L\omega_z \\ V_4 = \frac{1}{\sqrt{2}}(v_x - v_y) + L\omega_z \end{cases}$$

2.2 前向运动学（轮速 → 速度）

输入：四个轮子的实际速度 (V_1, V_2, V_3, V_4)
输出：底盘实际速度 (v_x, v_y, ω_z)

矩阵形式：

$$\begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} = \frac{1}{4} \begin{bmatrix} \sqrt{2} & -\sqrt{2} & -\sqrt{2} & \sqrt{2} \\ \sqrt{2} & \sqrt{2} & -\sqrt{2} & -\sqrt{2} \\ \frac{1}{L} & \frac{1}{L} & \frac{1}{L} & \frac{1}{L} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix}$$

展开形式：

$$\begin{cases} v_x = \frac{1}{4\sqrt{2}}(V_1 - V_2 - V_3 + V_4) \\ v_y = \frac{1}{4\sqrt{2}}(V_1 + V_2 - V_3 - V_4) \\ \omega_z = \frac{1}{4L}(V_1 + V_2 + V_3 + V_4) \end{cases}$$

2.3 轮速与电机转速转换

轮速转电机转速：

$$\omega_{motor,i} = \frac{V_i \cdot N}{R}$$

电机转速转轮速：

$$V_i = \frac{\omega_{motor,i} \cdot R}{N}$$

从编码器增量计算轮速：

$$V_i = \frac{2\pi R \cdot \Delta encoder_i}{PPR \cdot N \cdot \Delta t}$$

其中：

- R：轮子半径
- N：减速比
- PPR：编码器每转脉冲数
- Δt：采样周期（秒）

三、典型运动模式

3.1 前进运动

目标：$v_x = v$，$v_y = 0$，$\omega_z = 0$

$$\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = \frac{v}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix}$$

3.2 侧向平移

目标：$v_x = 0$，$v_y = v$，$\omega_z = 0$

$$\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = \frac{v}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$

3.3 原地旋转

目标：$v_x = 0$，$v_y = 0$，$\omega_z = \omega$

$$\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = L\omega \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

四、速度约束

4.1 单轮速度限制

最大轮速：V_{max}

最大前进速度：

$$v_{x,max} = \sqrt{2} \cdot V_{max}$$

最大旋转角速度：

$$\omega_{z,max} = \frac{V_{max}}{L}$$

4.2 速度缩放

当计算出的轮速超限时：

$$k = \frac{V_{max}}{\max(|V_1|, |V_2|, |V_3|, |V_4|)}$$

$$\begin{bmatrix} v'_x \\ v'_y \\ \omega'_z \end{bmatrix} = k \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix}$$

五、C++代码实现

5.1 运动学类实现

```cpp
#include <cmath>
#include <algorithm>

struct ChassisVelocity {
    double vx;  // m/s
    double vy;  // m/s
    double wz;  // rad/s
};

struct WheelSpeeds {
    double v1, v2, v3, v4;  // m/s
};

struct Pose {
    double x;      // m
    double y;      // m
    double theta;  // rad
};

class OmniKinematics {
private:
    const double L;              // 轮心到中心距离 (m)
    const double R;              // 轮子半径 (m)
    const double N;              // 减速比
    const double V_MAX;          // 最大轮速 (m/s)
    const double SQRT2_INV;      // 1/√2
    const double RAD_TO_RPM;     // 60/(2π)

public:
    OmniKinematics(double wheel_base, double wheel_radius,
                   double gear_ratio, double max_speed)
        : L(wheel_base), R(wheel_radius), N(gear_ratio),
          V_MAX(max_speed),
          SQRT2_INV(1.0 / std::sqrt(2.0)),
          RAD_TO_RPM(60.0 / (2.0 * M_PI)) {}

    // 逆向运动学：底盘速度 → 轮速
    WheelSpeeds inverseKinematics(const ChassisVelocity& cmd, bool limit = true) {
        WheelSpeeds wheels;

        wheels.v1 = SQRT2_INV * ( cmd.vx + cmd.vy) + L * cmd.wz;
        wheels.v2 = SQRT2_INV * (-cmd.vx + cmd.vy) + L * cmd.wz;
        wheels.v3 = SQRT2_INV * (-cmd.vx - cmd.vy) + L * cmd.wz;
        wheels.v4 = SQRT2_INV * ( cmd.vx - cmd.vy) + L * cmd.wz;

        // 速度限幅
        if (limit) {
            double max_speed = std::max({std::abs(wheels.v1), std::abs(wheels.v2),
                                         std::abs(wheels.v3), std::abs(wheels.v4)});
            if (max_speed > V_MAX) {
                double scale = V_MAX / max_speed;
                wheels.v1 *= scale;
                wheels.v2 *= scale;
                wheels.v3 *= scale;
                wheels.v4 *= scale;
            }
        }

        return wheels;
    }

    // 前向运动学：轮速 → 底盘速度
    ChassisVelocity forwardKinematics(const WheelSpeeds& wheels) {
        ChassisVelocity vel;

        double coeff = SQRT2_INV / 4.0;

        vel.vx = coeff * ( wheels.v1 - wheels.v2 - wheels.v3 + wheels.v4);
        vel.vy = coeff * ( wheels.v1 + wheels.v2 - wheels.v3 - wheels.v4);
        vel.wz = (wheels.v1 + wheels.v2 + wheels.v3 + wheels.v4) / (4.0 * L);

        return vel;
    }
```

```
    // 轮速 ↔ 电机转速 (rad/s)
    void wheelToMotorSpeed(const WheelSpeeds& wheels, double motor_speeds[4]) {
        motor_speeds[0] = wheels.v1 * N / R;
        motor_speeds[1] = wheels.v2 * N / R;
        motor_speeds[2] = wheels.v3 * N / R;
        motor_speeds[3] = wheels.v4 * N / R;
    }

    WheelSpeeds motorToWheelSpeed(const double motor_speeds[4]) {
        WheelSpeeds wheels;
        wheels.v1 = motor_speeds[0] * R / N;
        wheels.v2 = motor_speeds[1] * R / N;
        wheels.v3 = motor_speeds[2] * R / N;
        wheels.v4 = motor_speeds[3] * R / N;
        return wheels;
    }

    // 轮速 ↔ 电机转速 (RPM)
    void wheelToMotorRPM(const WheelSpeeds& wheels, double motor_rpm[4]) {
        motor_rpm[0] = wheels.v1 * N * RAD_TO_RPM / R;
        motor_rpm[1] = wheels.v2 * N * RAD_TO_RPM / R;
        motor_rpm[2] = wheels.v3 * N * RAD_TO_RPM / R;
        motor_rpm[3] = wheels.v4 * N * RAD_TO_RPM / R;
    }

    WheelSpeeds motorRPMToWheel(const double motor_rpm[4]) {
        WheelSpeeds wheels;
        wheels.v1 = motor_rpm[0] * R / (N * RAD_TO_RPM);
        wheels.v2 = motor_rpm[1] * R / (N * RAD_TO_RPM);
        wheels.v3 = motor_rpm[2] * R / (N * RAD_TO_RPM);
        wheels.v4 = motor_rpm[3] * R / (N * RAD_TO_RPM);
        return wheels;
    }

    // 编码器增量 → 轮速
    WheelSpeeds encoderToWheelSpeed(const int encoder_delta[4], double dt, int ppr) {
        WheelSpeeds wheels;
        double coeff = (2.0 * M_PI * R) / (ppr * N * dt);

        wheels.v1 = encoder_delta[0] * coeff;
        wheels.v2 = encoder_delta[1] * coeff;
        wheels.v3 = encoder_delta[2] * coeff;
        wheels.v4 = encoder_delta[3] * coeff;

        return wheels;
    }
};
```

5.2 里程计类实现

```
class Odometry {
private:
    Pose pose;
    OmniKinematics* kinematics;

    // 角度归一化到 [-π, π]
    void normalizeAngle(double& angle) {
        while (angle > M_PI) angle -= 2.0 * M_PI;
        while (angle < -M_PI) angle += 2.0 * M_PI;
    }

public:
    Odometry(OmniKinematics* kin)
        : pose{0.0, 0.0, 0.0}, kinematics(kin) {}

    // 更新位姿（从编码器）
    void update(const int encoder_delta[4], double dt, int ppr) {
        // 编码器 → 轮速 → 底盘速度
        WheelSpeeds wheels = kinematics->encoderToWheelSpeed(encoder_delta, dt, ppr);
        ChassisVelocity vel = kinematics->forwardKinematics(wheels);

        updatePose(vel, dt);
    }

    // 更新位姿（从底盘速度）
    void updatePose(const ChassisVelocity& vel, double dt) {
        // 转换到全局坐标系
        double cos_theta = std::cos(pose.theta);
        double sin_theta = std::sin(pose.theta);

        double vx_global = vel.vx * cos_theta - vel.vy * sin_theta;
        double vy_global = vel.vx * sin_theta + vel.vy * cos_theta;

        // 位姿积分
        pose.x += vx_global * dt;
        pose.y += vy_global * dt;
        pose.theta += vel.wz * dt;

        normalizeAngle(pose.theta);
    }

    Pose getPose() const { return pose; }

    void resetPose(double x = 0.0, double y = 0.0, double theta = 0.0) {
        pose.x = x;
        pose.y = y;
        pose.theta = theta;
    }
};
```

5.3 使用示例

```
#include <cstdio>

int main() {
    // 初始化参数
    double L = 0.3575;      // 轮心到中心距离 (m)
    double R = 0.0855;      // 轮子半径 (m)
    double N = 19.0;        // 减速比
    double V_max = 1.8;     // 最大轮速 (m/s)
    int PPR = 1024;         // 编码器分辨率

    OmniKinematics kinematics(L, R, N, V_max);
    Odometry odom(&kinematics);

    printf("=== 逆向运动学示例 ===\n");
    ChassisVelocity cmd = {0.5, 0.3, 0.2};  // vx, vy, wz

    WheelSpeeds wheels = kinematics.inverseKinematics(cmd);
    printf("底盘速度: vx=%.2f vy=%.2f wz=%.2f\n", cmd.vx, cmd.vy, cmd.wz);
    printf("轮速: V1=%.3f V2=%.3f V3=%.3f V4=%.3f m/s\n\n",
           wheels.v1, wheels.v2, wheels.v3, wheels.v4);

    double motor_rpm[4];
    kinematics.wheelToMotorRPM(wheels, motor_rpm);
    printf("电机转速: M1=%.1f M2=%.1f M3=%.1f M4=%.1f RPM\n\n",
           motor_rpm[0], motor_rpm[1], motor_rpm[2], motor_rpm[3]);

    printf("=== 前向运动学示例 ===\n");
    int encoder_delta[4] = {100, 120, 110, 90};
    double dt = 0.02;  // 20ms

    odom.update(encoder_delta, dt, PPR);
    Pose pose = odom.getPose();
    printf("编码器增量: E1=%d E2=%d E3=%d E4=%d\n",
           encoder_delta[0], encoder_delta[1], encoder_delta[2], encoder_delta[3]);
    printf("当前位姿: x=%.3f y=%.3f theta=%.2f°\n\n",
           pose.x, pose.y, pose.theta * 180.0 / M_PI);

    printf("=== 运动模式测试 ===\n");
    // 前进
    ChassisVelocity forward = {1.0, 0.0, 0.0};
    wheels = kinematics.inverseKinematics(forward);
    printf("前进: V1=%.3f V2=%.3f V3=%.3f V4=%.3f\n",
           wheels.v1, wheels.v2, wheels.v3, wheels.v4);

    // 侧移
    ChassisVelocity lateral = {0.0, 1.0, 0.0};
    wheels = kinematics.inverseKinematics(lateral);
    printf("侧移: V1=%.3f V2=%.3f V3=%.3f V4=%.3f\n",
           wheels.v1, wheels.v2, wheels.v3, wheels.v4);

    // 旋转
    ChassisVelocity rotation = {0.0, 0.0, 1.0};
    wheels = kinematics.inverseKinematics(rotation);
    printf("旋转: V1=%.3f V2=%.3f V3=%.3f V4=%.3f\n",
           wheels.v1, wheels.v2, wheels.v3, wheels.v4);

    return 0;
}
```

六、总结

6.1 核心公式

逆向运动学：

$$\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 & \sqrt{2}L \\ -1 & 1 & \sqrt{2}L \\ -1 & -1 & \sqrt{2}L \\ 1 & -1 & \sqrt{2}L \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix}$$

前向运动学：

$$\begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} = \frac{1}{4} \begin{bmatrix} \sqrt{2} & -\sqrt{2} & -\sqrt{2} & \sqrt{2} \\ \sqrt{2} & \sqrt{2} & -\sqrt{2} & -\sqrt{2} \\ \frac{1}{L} & \frac{1}{L} & \frac{1}{L} & \frac{1}{L} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix}$$

6.2 性能参数

运动模式	最大速度
前进/后退	$\sqrt{2} \cdot V_{max}$
侧向平移	$\sqrt{2} \cdot V_{max}$
原地旋转	$\frac{V_{max}}{L}$

---

robot #运动控制

四轮全向底底盘运动学分析
http://example.com/2025/12/08/四轮全向底底盘运动学分析/

作者	发布于	更新于	许可协议
EthanYLiang	2025年12月8日	2025年12月9日	